

Process Control Block (PCB) and Thread Control Block (TCB)

Internals, Data Structures and Relationship

Prashant Jadav

Fundamentals of Operating System Design (FOSD)

Learning Objectives

By the end of this topic, students should be able to:

- Explain the purpose of the Process Control Block (PCB).
- Identify and explain the major fields stored in a PCB.
- Explain the purpose and contents of the Thread Control Block (TCB).
- Distinguish process-level information from thread-level information.
- Explain how PCB and TCB are used during context switching.
- Analyze the relationship between a process and its threads.

Why Does the OS Need Control Blocks?

A running program needs much more information than just its executable code.

Operating system requirement

The OS must maintain the current state of every process and thread so that execution can be:

- suspended,
- scheduled,
- resumed,
- terminated, and
- managed independently.

Control blocks are kernel data structures that store this management information.

Process → PCB

Thread → TCB

Process Control Block (PCB)

Definition

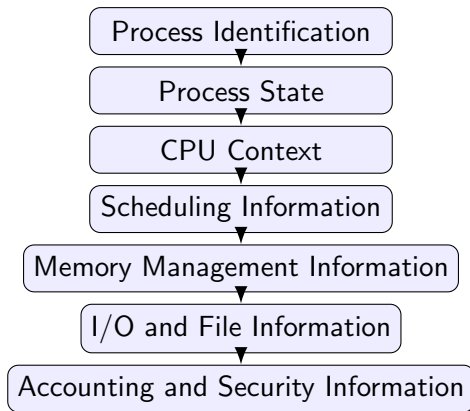
A **Process Control Block (PCB)** is a kernel data structure containing the information required by the operating system to manage and control a process.

The PCB is created when a process is created and is maintained until the process terminates and its resources are released.

Typical responsibilities

- Identify the process.
- Record its execution state.
- Store CPU-related context.
- Track scheduling information.
- Track memory and I/O resources.
- Maintain accounting and security information.

PCB: High-Level Structure



PCB Field 1: Process Identification

The OS needs identifiers to uniquely identify and relate processes.

- **PID:** Process Identifier.
- **PPID:** Parent Process Identifier.
- User/group identifiers associated with the process.
- Process credentials and ownership information.

Example

If process A creates process B:

$$PID_B = 2050, \quad PPID_B = PID_A$$

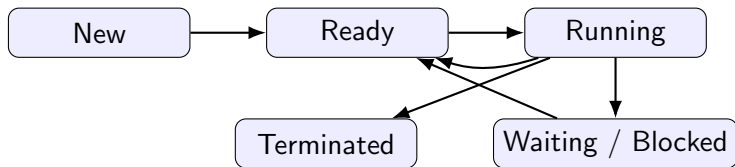
The parent-child relationship can therefore be represented by the kernel.

Important

A PID is an identifier; it is not the memory address of the PCB.

PCB Field 2: Process State

The PCB stores the current state of the process.



The state field helps the scheduler and other kernel components determine what can happen to the process next.

PCB Field 3: CPU Context

When a process is not executing, the OS must preserve enough CPU information to resume it later.

Typical CPU context includes:

- Program Counter (PC)
- Stack Pointer (SP)
- General-purpose registers
- Processor status / flags register
- Architecture-specific control registers

Context switching

During a context switch, the OS saves the outgoing execution context and restores the incoming process/thread context.

Running Context $\xrightarrow{\text{save}}$ PCB/TCB $\xrightarrow{\text{restore}}$ CPU

PCB Field 4: Scheduling Information

The PCB contains information used by the scheduler.

- Process priority.
- Scheduling class/policy.
- Queue pointers or scheduler links.
- CPU affinity information.
- Time-related scheduling information.
- Information used for accounting or fair scheduling.

Example

In priority scheduling, the scheduler may use a priority value stored in or associated with the process control information to determine which runnable entity should execute.

Design note

Modern operating systems may keep scheduler data in structures associated with the PCB rather than placing every field directly inside one contiguous PCB structure.

PCB Field 5: Memory Management Information

The PCB must identify the memory resources belonging to the process.

Examples

- Page-table base information.
- Address-space identifier.
- Memory region information.
- Code, data, heap and stack mappings.
- Memory protection information.

Virtual memory

The PCB provides the kernel with the information needed to associate a process with its virtual address space.

Process $\longrightarrow$ Virtual Address Space $\longrightarrow$ Page Tables $\longrightarrow$ Physical Memory

PCB Field 6: I/O and File Information

A process may have many resources associated with it.

- Open file descriptors / handles.
- References to files and directories.
- I/O status information.
- Devices or channels associated with operations.
- Pending I/O requests.

Example

When a process opens a file, the kernel maintains information that allows subsequent read/write operations to identify the corresponding open file object.

Important

The exact organization of I/O information is OS-dependent; it may be represented through references to other kernel objects.

PCB Field 7: Accounting and Security Information

The OS may maintain additional information for resource accounting and protection.

Accounting

- CPU time consumed
- Process start time
- Resource usage
- Scheduling statistics

Security

- User identity
- Group identity
- Credentials
- Privileges/capabilities
- Security context

Purpose

These fields support resource management, auditing, authorization and isolation.

PCB component	Typical information
Identification	PID, parent PID, user/group information
State	New, ready, running, blocked, terminated
CPU context	PC, SP, registers, processor status
Scheduling	Priority, policy, affinity, queue information
Memory	Address-space and page-table information
I/O	Open files, handles, devices, pending I/O
Accounting	CPU time, resource usage, timestamps
Security	Credentials, privileges, security context

What Is a Thread?

Definition

A **thread** is an independently schedulable sequence of execution within a process.

A process can contain one or multiple threads.

Threads in the same process generally share:

- Code
- Global/static data
- Heap
- Open resources
- Virtual address space

Each thread requires its own execution context.

Thread Control Block (TCB)

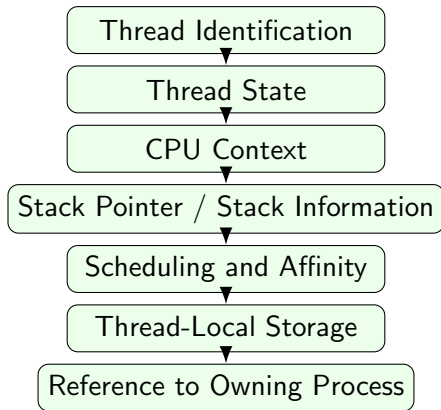
Definition

A **Thread Control Block (TCB)** is a kernel or runtime data structure containing information required to manage an individual thread.

Typical TCB information

- Thread identifier (TID).
- Thread state.
- Program counter.
- CPU register context.
- Stack pointer and thread stack information.
- Scheduling priority/policy.
- CPU affinity.
- Pointer/reference to the owning process.
- Thread-local storage information.

TCB: Internal Structure



TCB: CPU Context and Stack

Each thread must be able to resume its own execution.

Important thread-specific execution information

- Program Counter (PC)
- Stack Pointer (SP)
- General-purpose registers
- Processor status information
- Thread stack

Why separate stacks?

If multiple threads used the same execution stack, their function calls, local variables and return addresses would interfere with one another.

Thread 1 → Stack 1

Thread 2 → Stack 2

Thread 3 → Stack 3

Thread-Local Storage (TLS)

Some data should be private to an individual thread even though threads share a process address space.

Thread-Local Storage

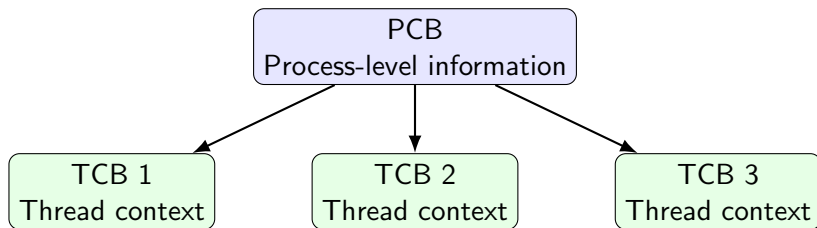
A mechanism that allows each thread to have its own instance of selected data.

Uses

- Per-thread error information.
- Thread-specific runtime state.
- Per-thread buffers.
- Library/runtime bookkeeping.

$$\text{Process} \rightarrow \begin{cases} \text{Thread 1} \rightarrow TLS_1 \\ \text{Thread 2} \rightarrow TLS_2 \\ \text{Thread 3} \rightarrow TLS_3 \end{cases}$$

PCB and TCB Relationship

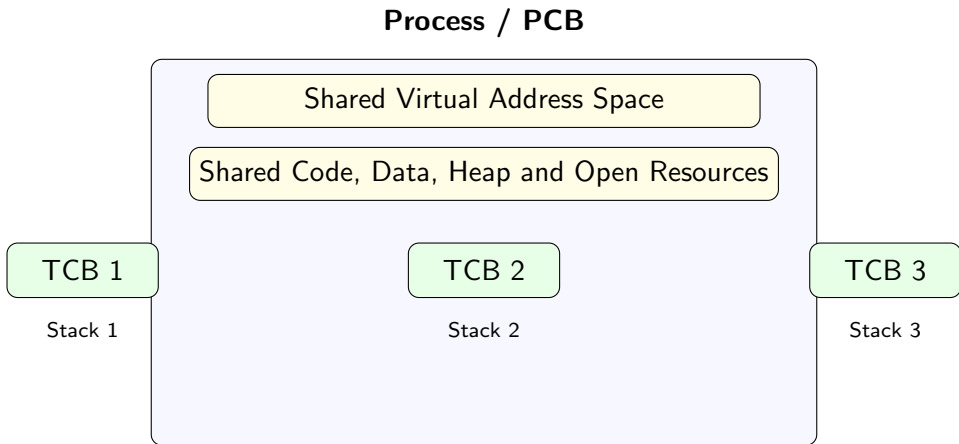


Threads share process resources such as address space, code, heap and open resources

What Belongs to PCB vs. TCB?

Information	PCB	TCB
Process ID	Yes	No
Thread ID	No	Yes
Process state	Yes	–
Thread state	–	Yes
Address space	Yes	Shared via process
Program counter	Process context / architecture-dependent	Yes
Registers	Process/thread context	Yes
Thread stack	No	Yes
Scheduling state	Process-related	Thread-related
Open files/resources	Associated with process	Usually shared/accessed through process
Thread-local storage	No	Yes
Parent-child relationship	Yes	No

Process with Multiple Threads



Context Switching: PCB/TCB Role

Context switch occurs when the CPU changes from one execution context to another.

- ① Save the current CPU context.
- ② Update the current thread/process state.
- ③ Select the next runnable thread.
- ④ Load the selected thread's saved CPU context.
- ⑤ Resume execution.



Important

In a multithreaded system, scheduling is commonly performed at the thread level, so the TCB is central to thread context switching.

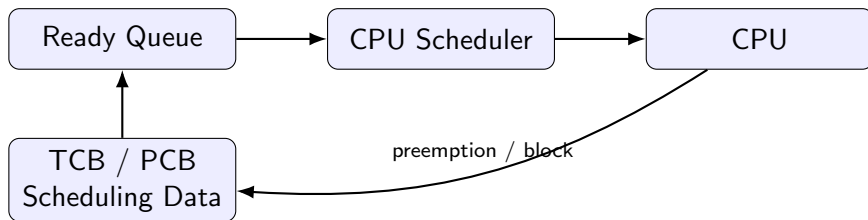
Process Switch vs. Thread Switch

	Thread switch within same process	Switch between processes
Execution context	Changes	Changes
Registers / PC / SP	Changes	Changes
Address space	Usually unchanged	Usually changes
Page-table context	Usually unchanged	May change
Shared resources	Remain available	Different process resources
Typical cost	Can be lower	Can be higher

Key observation

The cost of a context switch depends on the OS, hardware architecture, scheduling model and what state must be changed.

PCB/TCB and the Scheduler



- Scheduler selects a runnable execution entity.
- TCB/PCB-related information provides state and scheduling metadata.
- State changes are reflected in kernel scheduling structures.

Important Implementation Note

The terms **PCB** and **TCB** describe conceptual responsibilities.
Actual operating systems may organize the information differently.

- One kernel structure may contain multiple related fields.
- Some information may be stored in separate kernel objects.
- Scheduler structures may be separate from the main process structure.
- File descriptors may point to separate file-system objects.
- Memory-management information may be referenced through another structure.

Exam point

Do not assume that every OS has a single physical structure containing every PCB field. The PCB/TCB model is a conceptual representation of process and thread management information.

PCB and TCB: Key Differences

Aspect	PCB	TCB
Represents	Process	Thread
Identity	PID	TID
Address space	Owns/represents process address space	Shares process address space
CPU context	Associated with execution context	Thread-specific context
Stack	Process resources may include stacks	Own thread stack
Scheduling	Process-level information may exist	Thread scheduling information
Resources	Tracks process resources	Uses/shared process resources
Multiplicity	One per process	One or more per process

Higher-Order Thinking Questions

- ➊ A process contains four threads. Explain why the operating system needs four separate CPU contexts even though all threads share the same address space.
- ➋ Two threads of the same process have different program counters and stack pointers. Explain why this is necessary.
- ➌ During a context switch between two threads of the same process, which information can remain unchanged and which information must change? Justify.
- ➍ A process is terminated while it has three threads. Explain why the OS must manage both process-level and thread-level information during termination.

Numerical / Analytical Question

A process contains three threads:

T_1, T_2, T_3

Each thread has its own:

PC, SP, Registers, Stack

All three share:

Code, Data, Heap, Open Resources

Question

If T_1 is executing and the scheduler switches to T_2 :

- 1 Which TCB information must be saved for T_1 ?
- 2 Which TCB information must be restored for T_2 ?
- 3 Which process-level information normally remains unchanged?

Summary

Concept	Key idea
PCB	Stores information required to manage a process.
TCB	Stores information required to manage a thread.
CPU context	Includes PC, SP, registers and processor state.
Process resources	Address space, files and other resources are associated with the process.
Thread resources	Each thread has its own execution context and stack.
Context switch	Saves one execution context and restores another.
Scheduler	Uses process/thread state and scheduling information to select execution.

Key takeaway

PCB describes the process as a resource-owning entity; TCB describes an individual execution context within that process.

Process = Resource Ownership

Thread = Execution Unit

The PCB and TCB provide the kernel with the information needed to:

create — schedule — suspend — resume — synchronize — terminate

Understanding PCB and TCB is essential for understanding process management, thread management and context switching.